

Data Structures & Algorithms

-Ajay Iyengar

Email: iyengarajay@gmail.com

A little bit about myself

- 14+ years of experience in the Software industry
- Developer at heart
- Passionate about teaching
- Instructor on Udemy
- Blogger
- Love football

What is DSA, Really ?

- Data Structures = How you store data
- Algorithms = How to use that data to solve a problem efficiently
- Think of it like a kitchen:
 - **Data Structures** are the *containers* — fridge, drawers, shelves.
 - **Algorithms** are the *recipes* — the step-by-step way you prepare something delicious.
- Without organization (data structures), even the best chef (you) wastes time looking for ingredients.
- Without recipes (algorithms), your dish might end up... *weird*.

DSA is about organized thinking

Data Structures = How you organize your kitchen

Organized Kitchen:

```
Spices: Shelf A
Utensils: Drawer
Veggies: Fridge
```

Messy Kitchen:

```
Everything in
one big pile
on the counter
```

Cooking time: 30 mins

Cooking time: 2 hours
(spent searching!)

Algorithms = Your recipes

Good Recipe:

1. Boil water
2. Add pasta
3. Wait 10 mins
4. Drain
5. Serve

No Recipe:

Try random things
Hope it works
Burn the food
Start over
Give up, order pizza

The Java Connection

- You've already used **data structures** without realizing it:

```
List<String> names = new ArrayList<>();  
Map<Integer, String> students = new HashMap<>();
```

When you chose `ArrayList`, you chose a **data structure**

When you used `HashMap`, you benefited from a **hashing algorithm**

Combining Data & Logic

DSA in it's simplest form :

```
// Store 5 marks and find the average  
int[] marks = {90, 80, 75, 85, 95};  
int total = 0;  
for (int mark : marks) {  
    total += mark;  
}  
System.out.println("Average: " + total / marks.length);
```

- The array is the data structure (storage)
- The loop is like the algorithm (how we process it)

Why Learn DSA (Beyond Interviews)

Story 1: Priya's Accidental Performance Problem

Background: Priya, a DAC graduate, joined a startup as a Junior Developer. She learned Spring Boot, built REST APIs, and felt confident.

The Problem: She was asked to build a feature: "Check if user has already liked a post."

Why Learn DSA(Beyond Interviews)

The Problem: She was asked to build a feature: "Check if user has already liked a post."

Her Solution:

```
// Store liked posts  
List<Integer> likedPosts = new ArrayList<>(); // User has 10,000 liked posts  
  
// Check if user liked this post  
public boolean hasUserLiked(int postId) {  
    return likedPosts.contains(postId); // Checks all 10,000 posts!  
}
```

Why Learn DSA(Beyond Interviews)

- Worked fine in testing (user had 10 liked posts)
- In production: Active users had 10,000+ liked posts
- This method was called while loading feed (1000 posts per page)
- Result: 10,000 comparisons $\times$ 1000 posts = 10 million operations per page load!
- Page took many seconds to load Users complained: "App is so slow!"

```
// Store liked posts  
List<Integer> likedPosts = new ArrayList<>(); // User has 10,000 liked posts  
  
// Check if user liked this post  
public boolean hasUserLiked(int postId) {  
    return likedPosts.contains(postId); // Checks all 10,000 posts!  
}
```

The Fix (Not about optimization, about choosing appropriately):

```
// Use HashSet for "does this exist?" checks  
Set<Integer> likedPosts = new HashSet<>();  
  
public boolean hasUserLiked(int postId) {  
    return likedPosts.contains(postId); // Direct lookup, no searching!  
}
```

Important: DSA is NOT about making everything fast. It's about **understanding your tools** and **choosing appropriately**

Why Learn DSA(Beyond Interviews)

- **Story 2 :** Rahul's Code Review
- **Background:** Rahul had 2 years of experience. He considered himself a good developer.
- **The Situation:** He submitted code for review:

```
public boolean hasDuplicate(List<String> emails) {  
    for (int i = 0; i < emails.size(); i++) {  
        for (int j = i + 1; j < emails.size(); j++) {  
            if (emails.get(i).equals(emails.get(j))) {  
                return true;  
            }  
        }  
    }  
    return false;  
}
```

Why Learn DSA(Beyond Interviews)

- **The Situation:** He submitted code for review:
- Better Solution : Use a HashSet

```
public boolean hasDuplicate(List<String> emails) {  
    for (int i = 0; i < emails.size(); i++) {  
        for (int j = i + 1; j < emails.size(); j++) {  
            if (emails.get(i).equals(emails.get(j))) {  
                return true;  
            }  
        }  
    }  
    return false;  
}
```

```
public boolean hasDuplicate(List<String> emails) {  
    Set<String> seen = new HashSet<>();  
    for (String email : emails) {  
        if (!seen.add(email)) { // add() returns false if already exists  
            return true;  
        }  
    }  
    return false;  
}
```

With 10,000 emails, that's almost 50 million comparisons(worst case) : $n(n-1)/2$

Real-World Examples — Where DSA Hides in Plain Sight

App / System

Spotify Playlist

Undo/Redo in Word or Photoshop

OS Task Scheduler / Printer Queue

File Explorer / Database Index

Google Search Ranking

Hidden DSA Magic

Linked Lists (next/previous song navigation)

Stacks (reverse last action quickly)

Queues (tasks served in order of arrival)

Trees (B-Trees, Tries) (fast lookup and sorted storage)

Graphs + Priority Queues (PageRank and result ordering)

Where DSA Hides in Plain Sight

App / System

Hidden DSA Magic

Google Maps

Graphs! (cities are nodes, roads are edges; Dijkstra's algorithm finds shortest routes)

Netflix

Graphs + Matrices (user–movie relationships and recommendations)

Instagram Feed

Queues + Heaps (fetch latest posts, prioritize trending ones)

ChatGPT

Trees, Graphs, Dynamic Programming (language parsing, token prediction, context linking)

YouTube Recommendations

Graphs + Hash Maps (find similar videos and cache results fast)

© 2025 Ajay Iyengar. All rights reserved.

The Hidden Problem: Performance

Imagine two developers:

- ✓ **Dev A:** Solves a task in 1 second.
 - ✓ **Dev B:** Solves the same task in 10 seconds.
-
- Both are correct.
 - But at times in the real world, **speed matters**. (Be very careful !)
 - That's where DSA comes in. It teaches you to write **efficient** code, not just **working** code

How many checks if we have 1 million records ?

linear search vs. binary search

```
// Linear search -  $O(n)$ 
boolean found = false;
for (int i = 0; i < arr.length; i++) {
    if (arr[i] == target) {
        found = true;
        break;
    }
}
```

```
// Binary search -  $O(\log n)$ 
int low = 0, high = arr.length - 1;
while (low <= high) {
    int mid = (low + high) / 2;
    if (arr[mid] == target) {
        System.out.println("Found!");
        break;
    } else if (arr[mid] < target) {
        low = mid + 1;
    } else {
        high = mid - 1;
    }
}
```

DSA is what makes the difference: Exponential drop from 1,000,000 to ~20!

How DSA Shapes your Thinking

You start to:

- Break big problems into smaller sub-problems. (Recursion)
- Think about trade-offs: time vs. space.
- Visualize data flow.
- See patterns everywhere (trees, graphs, queues).

That's why top product companies test DSA. They don't want memorized syntax; they want **problem solvers**.

The Optimization Trap(When NOT to Optimize)

The Golden Rule

Premature optimization is the root of all evil in programming

- Donald Knuth

The Right Priority Order

1. Make it **WORK** (Correctness)
 - └ Does it solve the problem correctly?
2. Make it **CLEAR** (Readability)
 - └ Can teammates understand it in 6 months?
3. Make it **MAINTAINABLE** (Good structure)
 - └ Easy to modify when requirements change?
4. Make it **FAST** (Performance)
 - └ **ONLY** if measurements show it's needed

DSA IS

NOT ABOUT :

- ✗ Memorizing code solutions
- ✗ Solving trick questions
- ✗ Being a "genius" coder
- ✗ Making everything "optimal"
- ✗ Just passing interviews
- ✗ Writing the fastest code always

ABOUT :

- ✓ **Understanding your tools** - What each data structure does
- ✓ **Choosing appropriately** - Right tool for the problem
- ✓ **Thinking systematically** - Breaking problems into steps
- ✓ **Recognizing patterns** - "This is similar to..."
- ✓ **Making informed decisions** - Understanding trade-offs
- ✓ **Explaining your choices** - Why you chose X over Y
- ✓ **Knowing when to optimize** - Context matters

Arrays

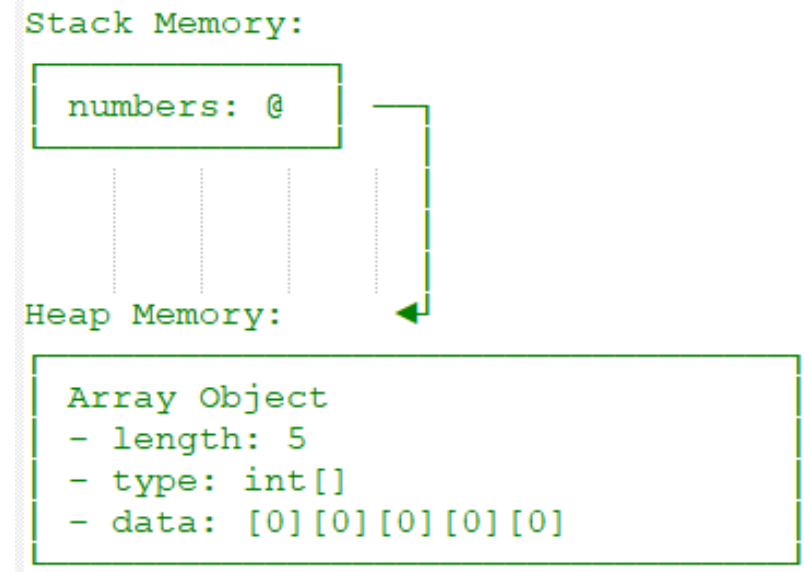
Arrays – The Foundation

- What happens when you declare an Array ?

```
int[] numbers = new int[5];
```

****Step 1: Reference Variable Creation (Stack)****

****Step 2: Array Object Creation (Heap)****



****Key Insight:**** The array reference lives on the stack, but the array data lives on the heap.

Arrays – Continuous memory & Cache Locality

- Arrays store elements in ****contiguous memory blocks****. This is why arrays are incredibly fast!

Memory Address	Value
0x1000	→ [10] ← arr[0]
0x1004	→ [20] ← arr[1]
0x1008	→ [30] ← arr[2]
0x100C	→ [40] ← arr[3]
0x1010	→ [50] ← arr[4]

- Why It Matters:-
 - CPU Cache: When you access `arr[0]`, the CPU loads nearby memory into cache
 - Predictable Access: `arr[i]` address = `base\_address + (i \* element\_size)`
 - No Pointer Chasing: Unlike linked structures, no following references

Arrays – Stack vs. Heap allocation

```
public void arrayExample() {  
    int[] arr1 = new int[3];    // Reference on stack, array on heap  
    arr1[0] = 10;  
  
    int[] arr2 = arr1;          // Copy reference, NOT the array!  
    arr2[0] = 20;  
  
    System.out.println(arr1[0]); // Prints 20! Both point to same array  
}
```

Stack:

arr1: @

arr2: @

Heap:

[20][0][0]
length: 3

****Important:** Both references point to the SAME array object!**

Array of Primitives vs. Array of Objects

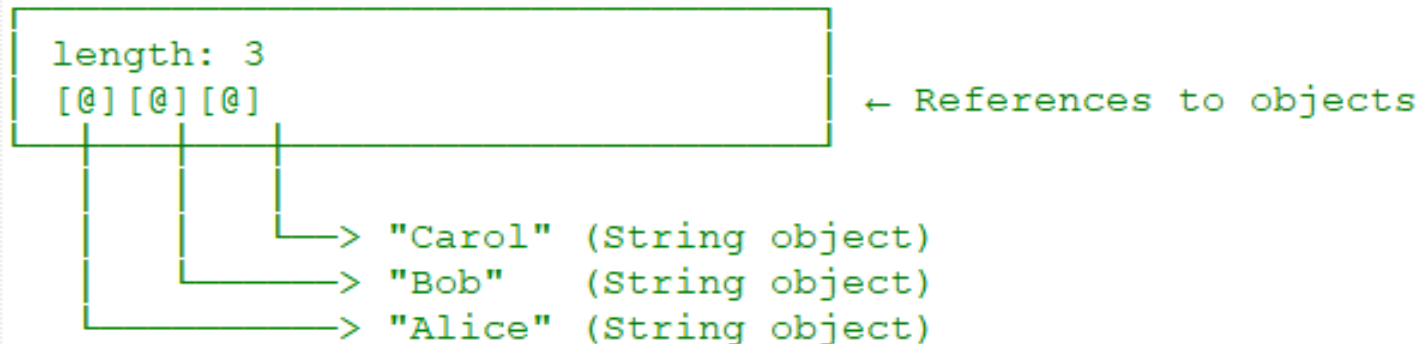
- Primitive Array | `int[] numbers = {10, 20, 30};`

Memory Layout:



- Object Array | `String[] names = {"Alice", "Bob", "Carol"};`

Memory Layout:



Object Array : Heap

```
String[] names = {"Alice", "Bob", "Carol"};
```

Object arrays store references,
not the objects themselves!

```
[String[] object]
type: String[]
length: 3
-----
index 0 → [String "Alice"]
index 1 → [String "Bob"]
index 2 → [String "Carol"]

[String "Alice"]
  value → ['A', 'l', 'i', 'c', 'e'] ← char[] on heap
  hash = (cached)
  coder = (internal flag)

[String "Bob"]
  value → ['B', 'o', 'b']

[String "Carol"]
  value → ['C', 'a', 'r', 'o', 'l']
```

Object Array : Heap

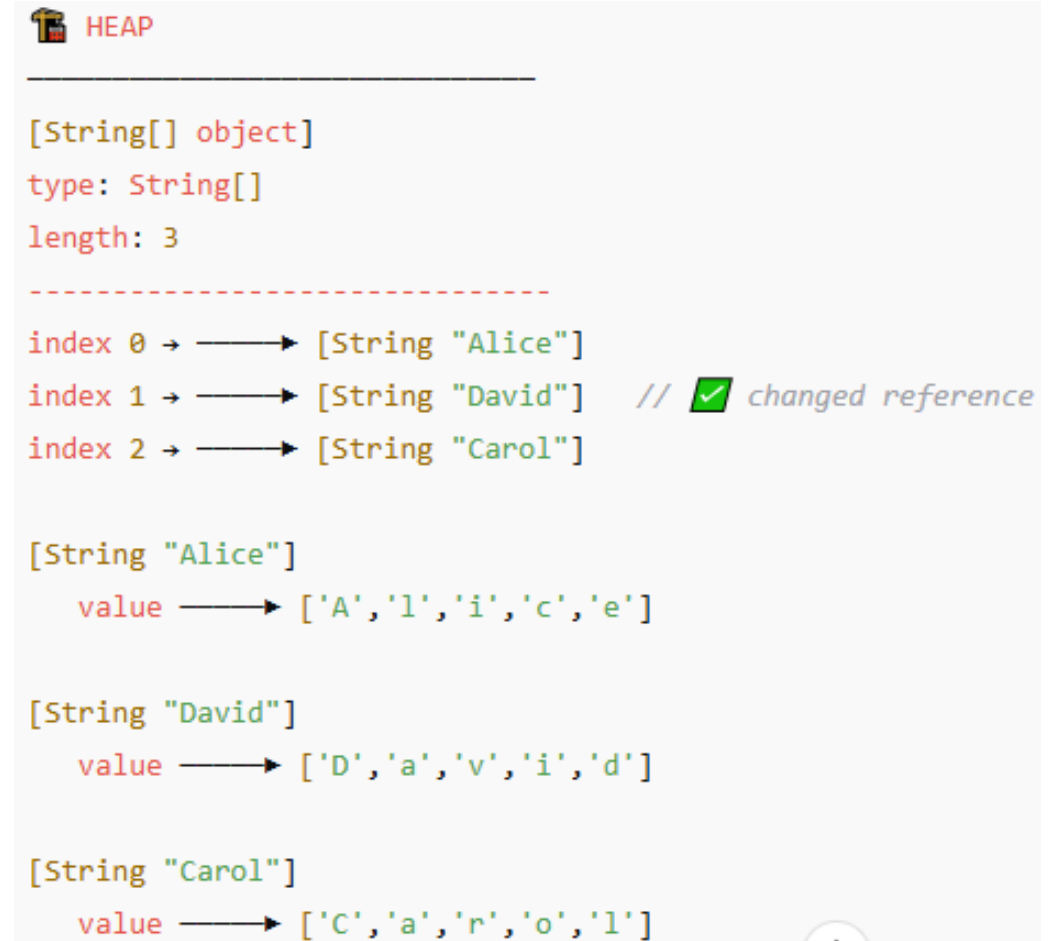
```
String[] names = {"Alice", "Bob", "Carol"};  
names[1] = "David";
```

String "Bob" may still exist in
heap temporarily, but is
unreachable — GC target

So what did we actually change?

Just a pointer inside the array — not the
objects themselves.

Arrays store addresses, not the actual strings.”



Java is ALWAYS pass-by-value

```
public void cannotChangeReference(int[] arr) {  
    arr = new int[]{100, 200}; // Creates NEW array, local reference only  
}  
  
public void canChangeContents(int[] arr) {  
    arr[0] = 100; // Modifies the SAME array  
}  
  
public static void main(String[] args) {  
    int[] numbers = {1, 2, 3};  
  
    cannotChangeReference(numbers);  
    System.out.println(numbers[0]); // Still 1  
  
    canChangeContents(numbers);  
    System.out.println(numbers[0]); // Now 100  
}
```

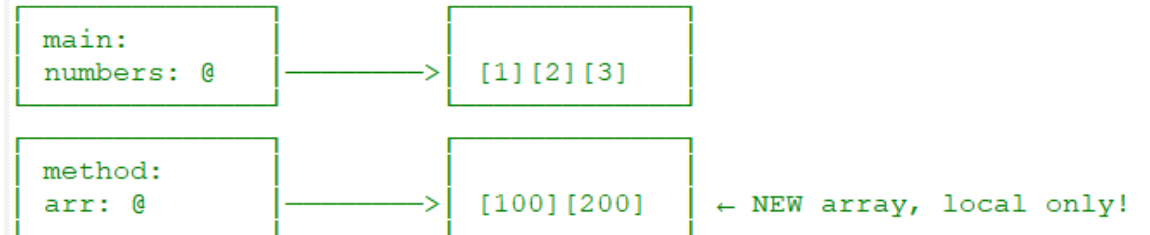
ALWAYS pass-by-value : Even for Arrays

```
public void cannotChangeReference(int[] arr) {  
    arr = new int[]{100, 200}; // Creates NEW array, local reference only  
}  
  
public void canChangeContents(int[] arr) {  
    arr[0] = 100; // Modifies the SAME array  
}  
  
public static void main(String[] args) {  
    int[] numbers = {1, 2, 3};  
  
    cannotChangeReference(numbers);  
    System.out.println(numbers[0]); // Still 1  
  
    canChangeContents(numbers);  
    System.out.println(numbers[0]); // Now 100  
}
```

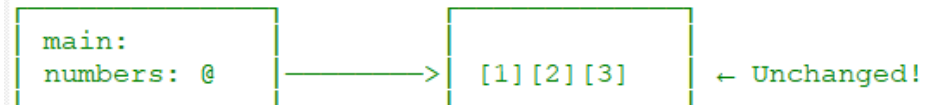
Before cannotChangeReference():



During cannotChangeReference():

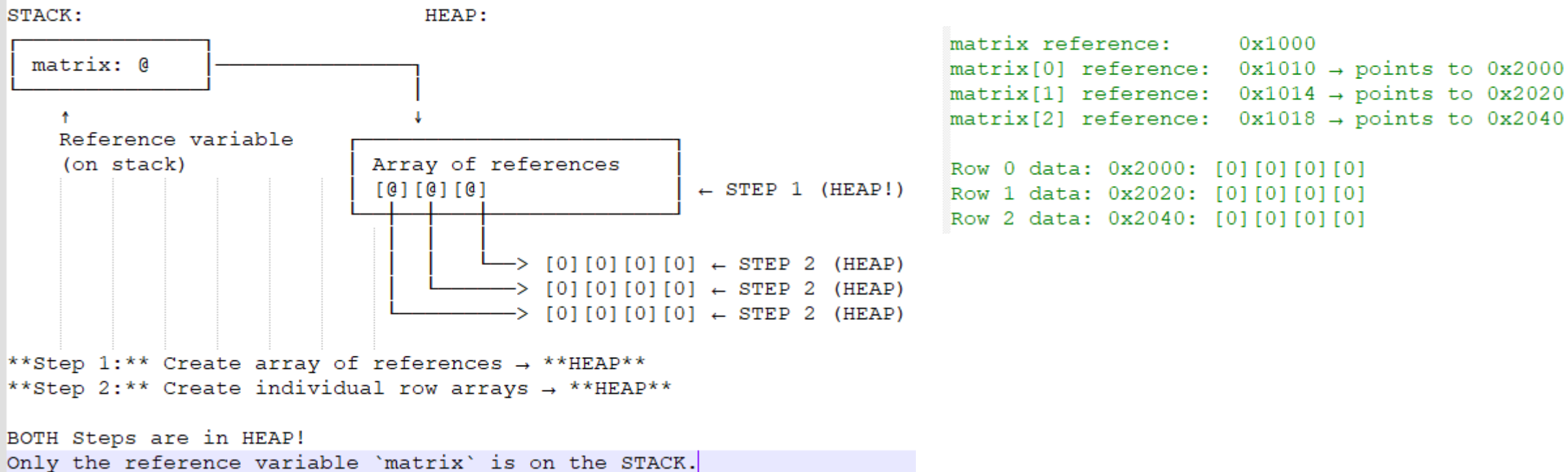


After method returns:



2D Arrays - Fundamentals

```
int[ ][ ] matrix = new int[3][4];
```



Jagged Arrays

- Rows can have different lengths

```
int[][] jagged = new int[3][];  
jagged[0] = new int[2];  
jagged[1] = new int[4];  
jagged[2] = new int[3];
```

- Visualization

[0] [0]

[0] [0] [0] [0]

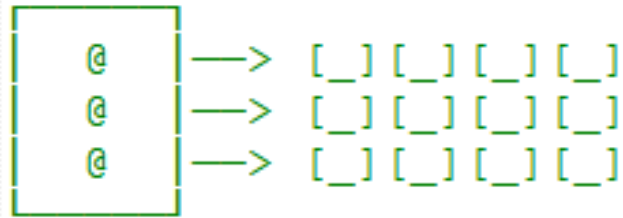
[0] [0] [0]

Memory Comparison

Rectangular vs. Jagged

- Use Cases:
 - Rectangular : Images, game boards, spreadsheets
 - Jagged : Triangle structures, variable-length data, Pascal's triangle

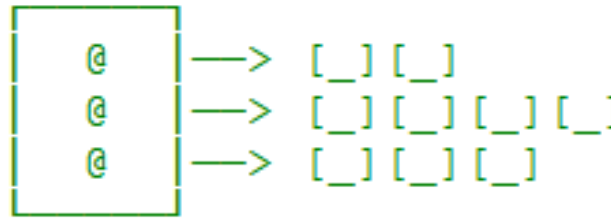
Rectangular:



Fixed memory

$3 * 4 = 12$ elements

Jagged:



Variable memory

$2 + 4 + 3 = 9$ elements

2D Array : Mental Model

- Visual Aid

`matrix[1][2] = 12` (row 1, col 2)

	j=0	j=1	j=2	j=3
i=0	00	01	02	03
i=1	10	11	12	13
i=2	20	21	22	23

`matrix[1][2] = 12` (row 1, col 2)

- `matrix[row][col]` | `matrix[i][j]`
- Think of it like addressing a house:-
 - i (row) = Which street?
 - j (col) = Which house number on that street?

How Many Objects Are Created When You Create a 2D Array?

```
int[][] arr = new int[3][4];
```

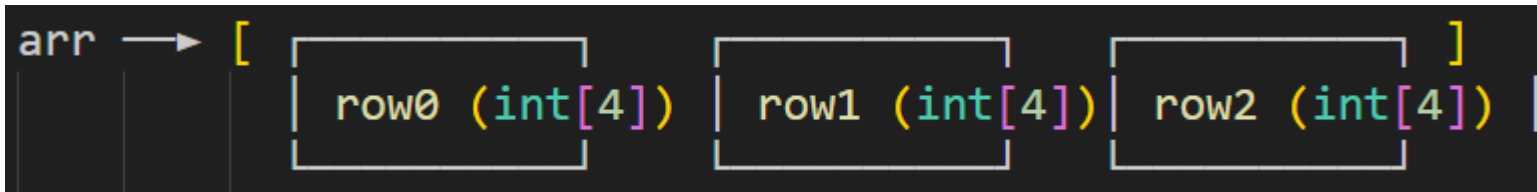
- ✓ **1 object for the outer array**
 - This is the array with **3 slots**, where each slot will hold a 1D array.
 - Think of it like: [row0 | row1 | row2]
- ✓ **3 objects for the 3 inner arrays**
 - Each row is an *independent* 1D array of size 4.
 - new int[4]
 - new int[4]
 - new int[4]
- **Total objects = 4** (1 outer array & 3 inner arrays)

```
1. Outer array object → new int[3][]  
2. Inner row objects:  
   arr[0] → new int[4]  
   arr[1] → new int[4]  
   arr[2] → new int[4]
```

Creating a 2D Array : What happens to columns?

```
int[][] arr = new int[3][4];
```

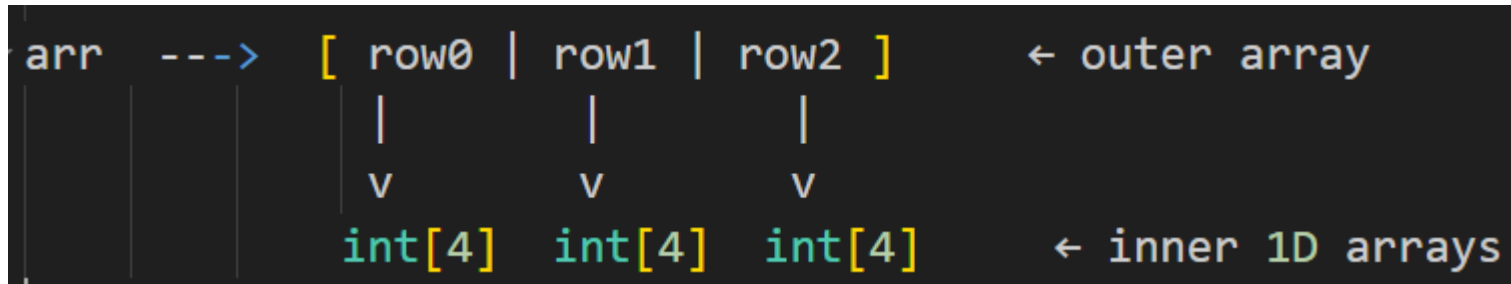
- Columns do not create separate objects
 - They are simply positions inside each row array
 - They don't exist as standalone objects
- Each `int[4]` is one object.
- Columns are just **indexes inside these 1D arrays**, not separate objects



2D Array : Array of Arrays

```
int[][] arr = new int[3][4];
```

- An array that contains 3 references
- 3 separate 1D arrays (size 4 in this case)



2D Array

Common Traversal Patterns

- Row-wise(Standard)

```
for (int i = 0; i < rows; i++) {  
    for (int j = 0; j < cols; j++) {  
        process(matrix[i][j]);  
    }  
}
```

Pattern:

```
→ → → →  
→ → → →  
→ → → →
```

- Column-wise

```
for (int j = 0; j < cols; j++) {  
    for (int i = 0; i < rows; i++) {  
        process(matrix[i][j]);  
    }  
}
```

Pattern:

```
↓ ↓ ↓ ↓  
↓ ↓ ↓ ↓  
↓ ↓ ↓ ↓
```

2D Array

Common Traversal Patterns

- Diagonal (Main)

```
for (int i = 0; i < Math.min(rows, cols); i++) {  
    process(matrix[i][i]);  
}
```

Pattern:

```
X _ _ _  
_ X _ _  
_ _ X _  
_ _ _ X
```

- Diagonal (Anti)

```
for (int i = 0; i < Math.min(rows, cols); i++) {  
    process(matrix[i][cols - 1 - i]);  
}
```

Pattern:

```
_ _ _ X  
_ _ X _  
_ X _ _  
X _ _ _
```

2D Array : Length vs. length

```
int[][] matrix = new int[3][4]; // 3 rows, 4 columns

matrix.length           // 3 (number of rows)
matrix[0].length        // 4 (number of columns in row 0)
matrix[i].length        // columns in row i

// Safe iteration
for (int i = 0; i < matrix.length; i++) {           // rows
    for (int j = 0; j < matrix[i].length; j++) {     // columns
        // Process matrix[i][j]
    }
}

matrix.length           → "How many arrays do I have?" → Rows
matrix[i].length        → "How long is this array?" → Columns
```

Array Myths Busted

Myth 1: Arrays are too limited, I should always use ArrayList "

Reality: Arrays are ****faster**** and use ****less memory**** when size is known.

```
// Scenario: Store 1 million sensor readings

// Option 1: Array
int[] readings = new int[1_000_000];
// Memory: ~4MB (just the data)
// Access: Direct, no method call overhead

// Option 2: ArrayList
List<Integer> readings = new ArrayList<>();
// Memory: ~12MB (boxing overhead + ArrayList overhead)
// Access: Method call + unboxing overhead

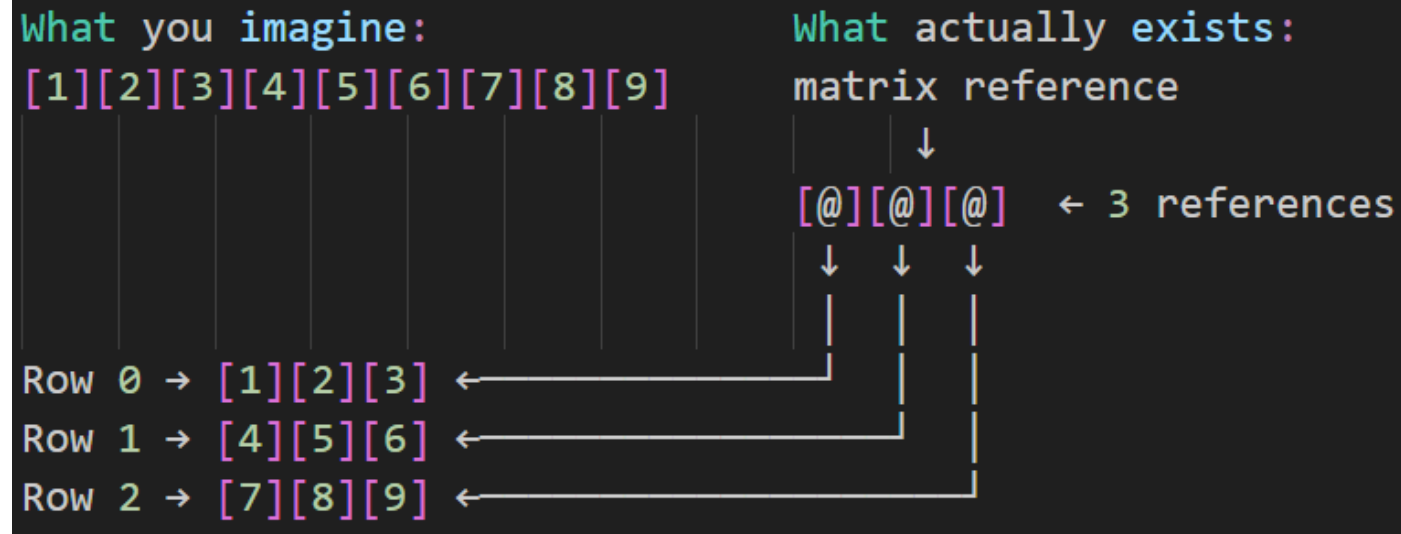
// Winner: Array
```

© 2025 Ajay Iyengar. All rights reserved.

Array Myths Busted

Myth 2: "2D arrays are stored as a contiguous block in memory "

```
int[][] matrix = {  
    {1, 2, 3},    // matrix[0] points to THIS array (row 0)  
    {4, 5, 6},    // matrix[1] points to THIS array (row 1)  
    {7, 8, 9}     // matrix[2] points to THIS array (row 2)  
};  
  
matrix[0]    → Gives you the entire first row: [1, 2, 3]  
matrix[0][1] → Gives you element at row 0, column 1: 2
```



Reality: They're ****arrays of references**** pointing to separate row arrays.

Array Myths Busted

Myth 3: "Arrays are initialized to zero, so I don't need to check "

```
int[] numbers = new int[5];  
System.out.println(numbers[0]); // 0 ✓  
  
String[] names = new String[5];  
System.out.println(names[0]); // null ✓  
System.out.println(names[0].length()); // NullPointerException!
```

Reality: Only ****primitive**** arrays are auto-initialized. Object arrays are null!

When to use...

Array

- Size known at compile time or initialization
- Performance critical code
- Primitive types (avoid boxing)
- Simple sequential access

ArrayList

- Size changes frequently
- Need utility methods (contains, indexOf, etc.)
- Don't mind slight overhead

Array Practice Problems

- **Problem 1 : Reverse Array In-Place**

- Statement : Given an integer array, reverse it **in-place** (without using extra space for another array).
- **Example 1:**
- Input: arr = [1, 2, 3, 4, 5] Output: arr = [5, 4, 3, 2, 1]
- **Example 2:**
- Input: arr = [10, 20, 30] Output: arr = [30, 20, 10]

Approach

- Use two pointers
- One starting from the beginning (left)
- One starting from the end (right)
- Swap elements and move pointers toward center
- Stop when pointers meet

Array Practice Problems

Problem 2 : Rotate Array Right by k Positions

Statement : Given an array and a number k, rotate the array to the **right** by k steps. Note: k can be larger than array length.

Example 1:

Input: arr = [1, 2, 3, 4, 5], k = 2

Output: arr = [4, 5, 1, 2, 3]

Explanation:

Rotate 1 step right: [5, 1, 2, 3, 4]

Rotate 2 steps right: [4, 5, 1, 2, 3]

Example 2:

Input: arr = [1, 2, 3, 4, 5, 6, 7], k = 3

Output: arr = [5, 6, 7, 1, 2, 3, 4]

Array Practice Problems

Problem 3 : Transpose a Matrix

Statement : Given a square matrix ($n \times n$), transpose it in-place. Transposing means converting rows to columns.

Note : Assume square matrix only.

```
Input: matrix = [  
  [1, 2, 3],  
  [4, 5, 6],  
  [7, 8, 9]  
]
```

```
Output: matrix = [  
  [1, 4, 7],  
  [2, 5, 8],  
  [3, 6, 9]  
]
```

```
Input: matrix = [  
  [1, 2],  
  [3, 4]  
]
```

```
Output: matrix = [  
  [1, 3],  
  [2, 4]  
]
```

Array Practice Problems

Problem 4 : Find Row and Column with Maximum Sum

Statement : Given a 2D matrix, find –

1. The row with the maximum sum
2. The column with the maximum sum. Return their indices (0-based).

Example 1 :

```
Input: matrix = [  
    [1, 2, 3],    // Row 0: sum = 6  
    [4, 5, 6],    // Row 1: sum = 15  
    [7, 8, 9]     // Row 2: sum = 24 (max row)  
]  
// Column sums: 12, 15, 18 (Column 2 is max)  
  
Output: maxRow = 2, maxCol = 2
```

Example 2 :

```
Input: matrix = [  
    [3, 7, 2],    // Row 0: sum = 12  
    [9, 1, 8]     // Row 1: sum = 18 (max row)  
]  
// Column sums: 12, 8, 10 (Column 0 is max)  
  
Output: maxRow = 1, maxCol = 0
```

Array Practice Problems

Problem 5 : Rotate Matrix 90 Degrees Clockwise

Statement : Given a square matrix (n x n), rotate it 90 degrees clockwise in-place.

```
Input:  matrix = [  
    [1, 2, 3],  
    [4, 5, 6],  
    [7, 8, 9]  
]
```

```
Output: matrix = [  
    [7, 4, 1],  
    [8, 5, 2],  
    [9, 6, 3]  
]
```

Visualization:

```
1 2 3      7 4 1  
4 5 6  →   8 5 2  
7 8 9      9 6 3
```

```
Input:  matrix = [  
    [1, 2],  
    [3, 4]  
]
```

```
Output: matrix = [  
    [3, 1],  
    [4, 2]  
]
```